

Callbacks, Higher-Order Functions & Closures — Short Study Chapter

Goal: One compact chapter (basics → advanced patterns) you can study tomorrow. Short, focused code examples and exercises.

1. Quick overview

- **First-class functions:** Functions are values. You can assign them to variables, pass them to other functions, return them.
- **Callback:** A function you pass into another function so it can be called later.
- **Higher-order function (HOF):** A function that **takes** a function as an argument and/or **returns** a function.
- **Closure:** An inner function that *remembers* variables from its outer scope even after the outer function has finished.

These four ideas are the primitives; everything that follows is a combination of them.

2. Minimal examples (the core 20%)

First-class function & callback

```
function greet(cb) {  
  cb('hello');  
}  
  
function print(msg) {  
  console.log(msg);  
}  
  
greet(print); // prints "hello"
```

HOF that returns a function (wrapper)

```
function withLogging(fn) {  
  return function(...args) {  
    console.log('calling', fn.name, 'with', args);  
    return fn(...args);  
  };  
}
```

```
const add = (a,b) => a + b;
const loggedAdd = withLogging(add);
loggedAdd(1,2); // logs call and returns 3
```

Closure (private state)

```
function counter() {
  let n = 0;
  return function() {
    n += 1;
    return n;
  };
}

const c = counter();
console.log(c()); // 1
console.log(c()); // 2
```

3. Asynchronous callbacks vs. Promises / async-await

Callback-based async (old style):

```
function readFile(path, cb) {
  fs.readFile(path, 'utf8', (err, data) => {
    if (err) return cb(err);
    cb(null, data);
  });
}
```

Problems: nested callbacks (aka "callback hell"), error-handling duplication, hard to compose.

Promise / async/await (modern):

```
function readFileP(path) {
  return new Promise((resolve, reject) => {
    fs.readFile(path, 'utf8', (err, data) => {
      if (err) reject(err);
      else resolve(data);
    });
  });
}
```

```

}

// or with util.promisify in Node

async function run() {
  try {
    const text = await readFileP('./file.txt');
    console.log(text);
  } catch (err) {
    console.error(err);
  }
}

```

Takeaway: Promises + async/await are built on callbacks but give cleaner composition and error flow.

4. Useful higher-order patterns (practical)

4.1 Wrappers / decorators (logging, timing)

```

function timeIt(fn) {
  return function(...args) {
    const t0 = Date.now();
    const res = fn(...args);
    const ms = Date.now() - t0;
    console.log(fn.name, 'took', ms, 'ms');
    return res;
  };
}

```

Works for sync functions. For async functions, return `await fn(...args)` inside an `async` wrapper.

4.2 Currying & partial application

```

function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn(...args);
    return (...rest) => curried(...args, ...rest);
  };
}

const sum3 = (a,b,c) => a + b + c;

```

```
const cur = curry(sum3);  
cur(1)(2)(3); // 6
```

4.3 Composition / pipe

```
const compose = (f, g) => x => f(g(x));  
// or for many: reduceRight
```

4.4 Memoize

```
function memoize(fn) {  
  const cache = new Map();  
  return function(...args) {  
    const key = JSON.stringify(args);  
    if (cache.has(key)) return cache.get(key);  
    const res = fn(...args);  
    cache.set(key, res);  
    return res;  
  };  
}
```

4.5 Debounce & Throttle (event control)

```
function debounce(fn, wait) {  
  let t;  
  return function(...args) {  
    clearTimeout(t);  
    t = setTimeout(() => fn(...args), wait);  
  };  
}
```

4.6 Retry with backoff

```
async function retry(fn, attempts = 3, delay = 200) {  
  let lastErr;  
  for (let i = 0; i < attempts; i++) {  
    try {  
      return await fn();  
    } catch (err) {  
      lastErr = err;  
      await new Promise(r => setTimeout(r, delay * (i+1)));  
    }  
  }  
}
```

```
    throw lastErr;
}
```

4.7 once (call only once)

```
function once(fn) {
  let done = false;
  let result;
  return function(...args) {
    if (done) return result;
    done = true;
    result = fn(...args);
    return result;
  };
}
```

5. Closures: deeper intuition & common uses

- **Closures capture references**, not copies. If the captured variable is an object, changes are visible.
- **Use-cases**: private state (counters, factories), memoization caches, partial application, data hiding (module pattern).
- **Memory note**: closures can keep large objects alive if referenced; be mindful in long-lived closures.

Example — module pattern:

```
const createStorage = () => {
  const store = {};
  return {
    set(k,v) { store[k] = v; },
    get(k) { return store[k]; }
  };
};

const s = createStorage();
s.set('a', 1);
```

6. Advanced control-flow / functional patterns

- **Trampoline**: technique to avoid stack overflows when using recursion by returning thunks and executing them iteratively.

- **Generators + async:** using generator runners (older libs) to write linear async code. Replaced in many cases by `async/await`.
 - **Decorators:** same idea as wrappers — in frameworks / TypeScript you see `@decorator` which wraps functions/classes.
-

7. React connections (practical)

- `useCallback(fn, deps)` returns a memoized reference — useful to avoid re-creating functions that cause re-renders. Under the hood it returns a stable function reference (similar idea to closures/HOFs).
- `useMemo` memoizes values (like memoize pattern).
- Custom hooks (like your `useFetch`) are HOF-like: they *take* a function (your API call) and *return* state and an action function (a closure that calls your API and updates state).

Pitfall: **stale closures** — if a closure captures state that later changes, the closure may still reference old values. In React this shows up with `setInterval` or event listeners; fix it by keeping fresh refs or using `deps` arrays correctly.

8. Common pitfalls & debugging tips

- **Callback hell:** break code into named functions or use Promises/async.
 - **Uncaught errors in async callbacks:** always handle errors. With Promises use `catch` or `try/catch` with `async/await`.
 - **Stale closures:** in frameworks, be careful when closures capture props/state that change later.
 - **Memory retention:** closures keep referenced variables alive — avoid capturing huge objects unnecessarily.
 - **this binding:** when using callbacks as methods, remember `.bind`, arrow functions, or pass bound function.
-

9. Exercises (practice)

1. Implement `once(fn)` and test it.
2. Implement `curry(fn)` and curry a 3-arg function.
3. Memoize a slow Fibonacci and measure speed-up.
4. Convert a callback-style function (`fs.readFile`) to return a Promise.
5. Write `compose` for two and for many functions.
6. Implement `debounce` and attach it to a simulated window `resize` handler.
7. Write `retry` that retries an async function with exponential backoff.
8. Recreate a simple `useFetch` that returns `{ data, loading, error, fn }` (manual) in a small React app.
9. Modify the hook to auto-fetch on mount and compare behavior.
10. Fix a stale closure bug: create a counter with `setInterval` that uses stale value; then fix with `useRef` or functional `setState`.

10. Quick cheat-sheet (one-line patterns)

- `withLogging(fn) => (...a) => { log(); return fn(...a); }`
- `memoize(fn) => cache => fast lookups`
- `debounce(fn, wait)` delays execution until quiet
- `throttle(fn, ms)` limits rate
- `curry(fn)` transform `f(a,b,c) -> f(a)(b)(c)`
- `compose(...fns)` right-to-left composition
- `once(fn)` run once and cache result
- `retry(fn)` attempt multiple times

Study plan for tomorrow (suggested)

1. 30 min — Read this chapter slowly, type each example into a REPL or node and run it.
2. 60–90 min — Do 3 exercises (pick 1, 3, and 6). Try to write tests or `console.time` checks.
3. 30 min — Build a tiny React page that uses `useFetch` (manual) and confirm understanding.
4. 15 min — Note three areas still unclear; ask me to explain them.

If you want, I can also: - generate solutions for the exercises, - export this as a printable PDF, - convert exercises into an interactive CodeSandbox (I can produce the files), - or create flashcards from the cheat-sheet.

Good luck tomorrow — you've already got the right instincts. 🍊